General

Generelt om prosjektarbeidet

- All prosjektarbeid i IND320 resulterer i personlige innleveringer og online-apper.
- En **Jupyter Notebook** som kjøres lokalt på datamaskinen din (senere med tilgang til online- og lokaldatabaser) vil være din grunnleggende utviklings- og dokumentasjonsplattform.
- Må inneholde en kort beskrivelse av **AI-bruk**.
- Må inneholde en **logg på 300–500 ord** som beskriver det obligatoriske arbeidet (inkludert både Jupyter Notebook- og Streamlit-erfaring).
- Må inneholde lenker til din offentlige **GitHub-repositorium** og **Streamlit-app** for det obligatoriske arbeidet.
- Dokumentets overskrifter skal være klare og enkle å navigere i under utvikling.
- Alle kodeblokker må inneholde nok kommentarer til å være forståelige og reproduserbare hvis noen arver prosjektet ditt.
- Alle kodeblokker må kjøres før eksport til PDF slik at meldinger og grafer vises. I tillegg skal `.ipynb`-filen legges til i GitHub-repositoriet der du har Streamlit-prosjektet ditt.
- En **Streamlit-app** som kjøres på `https://[yourproject].streamlit.app/`.
- Dette er en online-versjon av prosjektet som henter data fra MongoDB-databasen din.
- Koden, som hostes på GitHub, må inneholde relevante kommentarer fra Jupyter Notebook samt ytterligere kommentarer angående Streamlit-bruk.
- Det er fire deler i prosjektarbeidet som bygger på hverandre og resulterer i en endelig portefølje og app som presenteres ved slutten av semesteret.
- Samarbeid applauseres, og bruk av AI-verktøy oppmuntres.

Oppgaver (Tasks)

1. Kontoer og repositorier (Accounts and repositories)

- Gjenbruk kontoen, repositoriet og Streamlit-appen fra forrige del av prosjektarbeidet.
- Inntil fagfellevurdering (peer review) og tilbakemelding er fullført, bør du pushe til en midlertidig GitHub-branch for senere sammenslåing (merge).

2. Lokal database: Cassandra

- Hvis det ikke allerede er gjort, sett opp Cassandra og Spark som beskrevet i boka.
- Test at Spark-Cassandra-tilkoblingen fungerer.
- Cassandra-databasen vil bli nådd fra Jupyter Notebook og brukes til å lagre data fra API-ene nevnt senere.

3. Ekstern database: MongoDB

- Hvis det ikke allerede er gjort, klargjør en MongoDB-konto på `mongodb.com`.
- Test at du kan manipulere data fra Python.
- MongoDB-databasen vil lagre data som har blitt kuttet/kuratert/forberedt gjennom Jupyter Notebook og Spark-filtrering.
- Disse dataene nås direkte fra Streamlit-appen.

4. API

- Gjør deg kjent med API-et på **Magasinstatistikk API** og beskrivelsene hos **NVE Magasinstatistikk**.
- Husk erfaringene dine angående tidskolonnene fra del 1 av prosjektet.
- Identifiser områdene i Norge som dekkes av de forskjellige API-ene.
- Gjør deg kjent med API-et i **ENTSO-E API reference** og dets kontohåndtering og grafiske grensesnitt hos **ENTSO-E**.
- Opprett en konto og få en sikkerhetstoken.

5. Jupyter Notebook

- Fjern CSV-filen for Magasinstatistikk og bruk en direkte API-tilkobling i stedet.
- Gjenopprett de tilhørende grafene fra del 1 av prosjektet ved hjelp av API-tilkoblingen.
- Bruk ENTSO-E API-et til å hente overføringsdata på tvers av landegrenser fra/til Norge for de siste to årene.
- Sett inn dataene i Cassandra ved hjelp av Spark.
- Bruk Spark til å trekke ut relevante kolonner fra Cassandra.
- Lag følgende grafer:
  - Et kakediagram (pie chart) for den totale produksjonen for ett år fra en valgt tilkobling, der hver del av kaken er mengden inn og ut av Norge.
  - Et linjediagram (line plot) for den første måneden i et år for en valgt tilkobling. Lag egne linjer for hver tilkobling og retning.
- Sett de Spark-utdukkede dataene inn i din MongoDB.
- Husk å fylle ut loggen og AI-beskrivelsen nevnt i Generelt-seksjonen ovenfor.

6. Streamlit-app

- Oppdater Streamlit-appen din fra del 1 av prosjektet i henhold til følgende punkter:
  - Hvis du har noe du vil beholde på side 4, flytt det til en ny side 5.
  - Etabler en tilkobling til Magasinstatistikk-API-et og bruk dette i stedet for den nå slettede CSV-filen.
  - **Oppdatering 12.10:**
    - Valgglidebryteren (selection slider) som var på side 3, skal nå kontrollere nedlastingsintervallet (dvs. hvilke data som lastes ned).
    - Bruk radioknapper (`st.radio`) for å velge type områder (`EL`, `NO`, `VASS`) slik at grafer og `LineChartColumn()` viser tilhørende grafikk (f.eks. linjer) for hvert av de tilhørende areanumrene.
  - Reorganiser side 2 og 3 på den måten du finner mest logisk og brukervennlig. Bruk en ekstra side om du trenger det. Løsninger som avviker litt fra beskrivelsene ovenfor, men som forbedrer opplevelsen, er velkomne.
  - Etabler en tilkobling til MongoDB-databasen din. Når du kjører dette på `streamlit.io`, husk å kopiere dine hemmeligheter (secrets) til nettsiden i stedet for å eksponere dem på GitHub.
  - På en egen side, del visningen inn i to kolonner ved hjelp av `st.columns`.
    - På venstre side: bruk radioknapper (`st.radio`) for å velge en tilkobling og en annen kontroll for å velge år, og vis et kakediagram som i Jupyter Notebook.
    - På høyre side: bruk `st.pills` for å velge hvilke måneder som skal inkluderes, og vis et linjediagram som i Jupyter Notebook (men for et valgt sett med måneder).
  - Under kolonnene, sett inn en utvider (`st.expander`) der du kort dokumenterer kilden til dataene som vises på siden.

Links

- GitHub repository: https://github.com/IsmailNabi/IND320-prosjekt
- Streamlit app: https://ismail.streamlit.app

0. Setup

In [1]:

```python
# Imports and paths. The notebook lives in notebooks/, the shared modules in ../modules
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

ROOT = Path.cwd().parent                        # project root (relative, reproducible)
sys.path.insert(0, str(ROOT))

from modules import api, analysis, db
from modules.config import get_secrets
from modules.helpers import METRIC_LABELS, prepare_reservoir_df

# Credentials are read from No_sync/secrets.toml (git-ignored), see secrets.example.toml
SECRETS = get_secrets()
```

In [2]:

```python
# Test the MongoDB connection with the credentials from No_sync/secrets.toml
m = SECRETS["mongo"]
client = db.get_client(db.build_uri(m["user"], m["password"], m["host"]))
client.admin.command("ping")
print("MongoDB connection OK")
```

```
MongoDB connection OK
```

1. NVE Magasinstatistikk: direct API connection (replaces reservoirs.csv)

In [3]:

```python
# Download all weekly reservoir data and area names directly from the API
raw = api.fetch_reservoirs_raw()
areas = api.fetch_areas()

# Shared helpers rename columns, clean numbers and build dates (see modules/helpers.py)
df = analysis.add_area_names(prepare_reservoir_df(raw), areas)
print(df.shape)
display(df.head())
display(areas.groupby("area_type").size().rename("number of areas"))
```

```
(14922, 13)
```

	date	area_type	area_number	iso_year	iso_week	filling_degree	capacity_twh	filling_twh	next_publication_date	filling_degree_prev_week	change_filling_degree	month	nam
0	1995-01-08	EL	1	1995	1	0.617574	6.003264	3.707457	0001-01-01T00:00:00	0.583705	0.033869	1995-01	NO
1	1995-01-15	EL	1	1995	2	0.579205	6.003264	3.477117	0001-01-01T00:00:00	0.617574	-0.038369	1995-01	NO
2	1995-01-22	EL	1	1995	3	0.546713	6.003264	3.282063	0001-01-01T00:00:00	0.579205	-0.032491	1995-01	NO
3	1995-01-29	EL	1	1995	4	0.504545	6.003264	3.028916	0001-01-01T00:00:00	0.546713	-0.042168	1995-01	NO
4	1995-02-05	EL	1	1995	5	0.465335	6.003264	2.793528	0001-01-01T00:00:00	0.504545	-0.039210	1995-02	NO

```
area_type
EL      5
NO      1
VASS    4
Name: number of areas, dtype: int64
```

1.1 Areas covered by the NVE API

The API has four endpoints: `HentOffentligData` (weekly values since 1995), `HentOmråder` (area names), `HentOffentligDataSisteUke` (latest week only) and `HentOffentligDataMinMaxMedian` (min/max/median over 20 years). No API key is needed. The areas are:

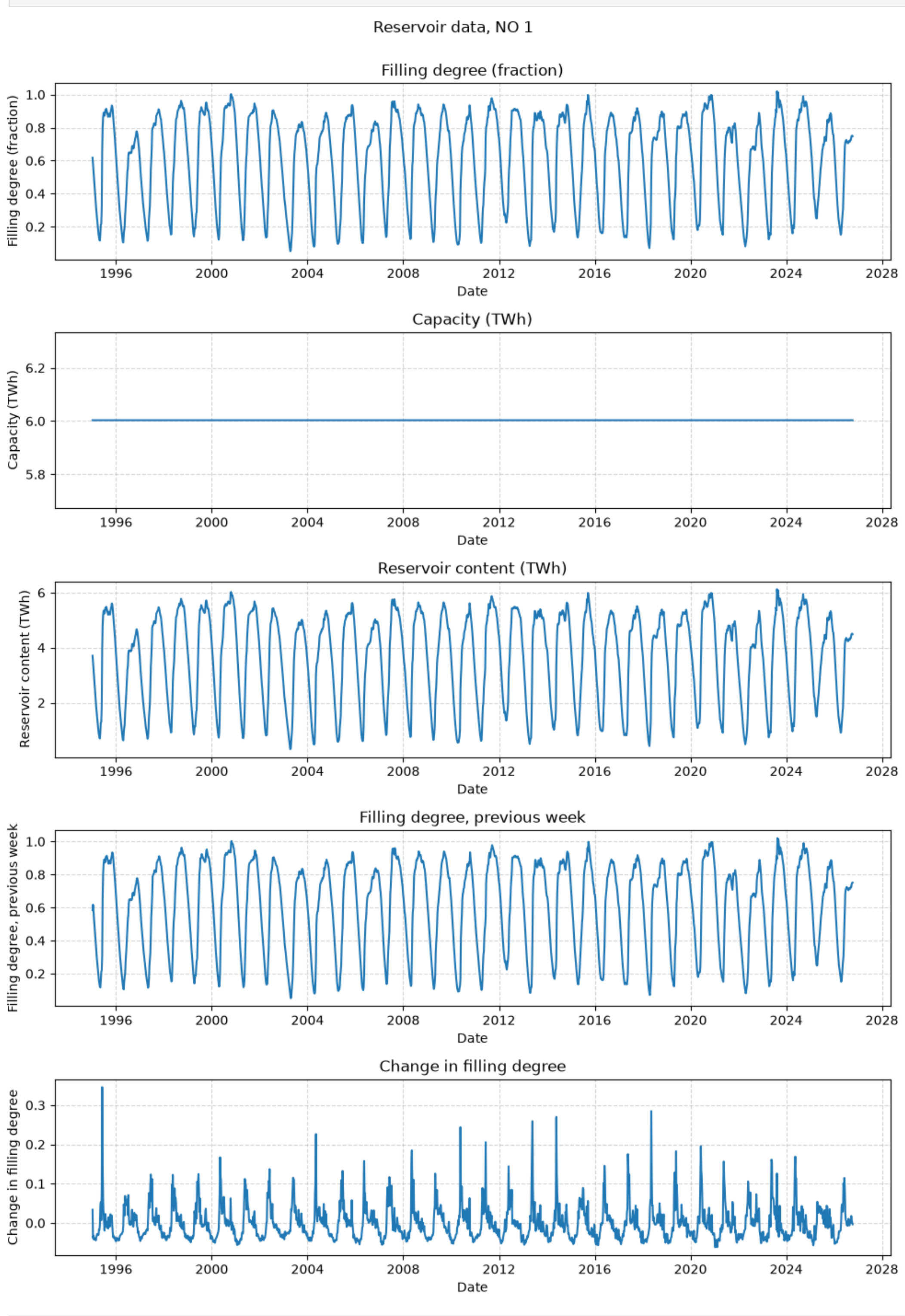
Type	Meaning	Number of areas
`NO`	Norway as a whole	1
`EL`	Electricity price areas (NO1-NO5)	5
`VASS`	Watercourse areas	4

The same week appears once per area, so a plot must always be limited to **one area** (or one line per area).

In [4]:

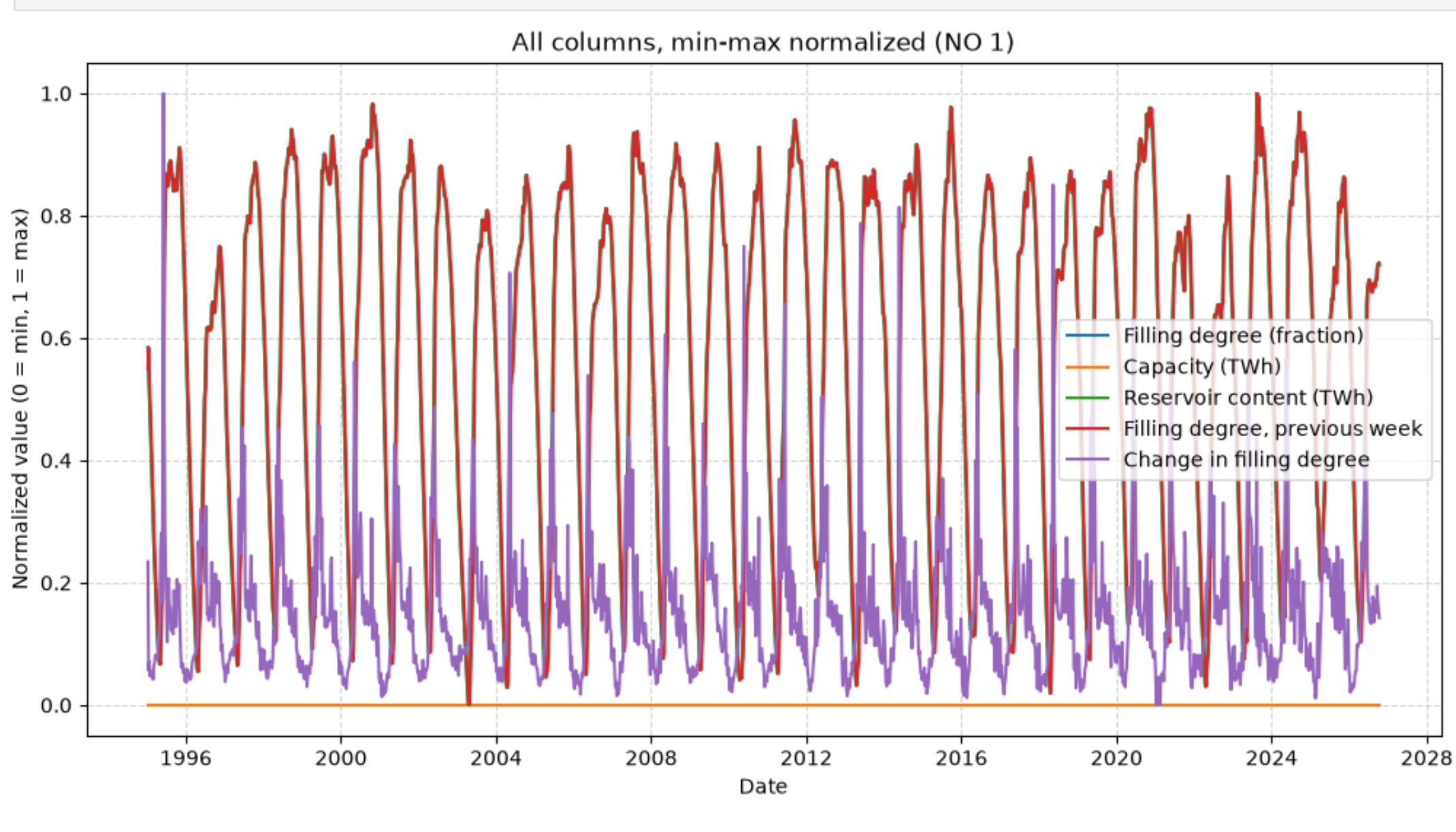
```python
# Restore the part 1 plots, now with real dates and for ONE chosen area
AREA_TYPE, AREA_NUMBER = "EL", 1
area_df = df[(df["area_type"] == AREA_TYPE) & (df["area_number"] == AREA_NUMBER)]
area_name = area_df["name"].iloc[0]

# One plot per column, each with date values on the x-axis
fig, axes = plt.subplots(len(METRIC_LABELS), 1, figsize=(10, 14))
for ax, (col, label) in zip(axes, METRIC_LABELS.items()):
    ax.plot(area_df["date"], area_df[col])
    ax.set_title(label)
    ax.set_xlabel("Date")
    ax.set_ylabel(label)
    ax.grid(True, linestyle="--", alpha=0.5)
fig.suptitle(f"Reservoir data, {area_name}", y=1.0)
plt.tight_layout()
plt.show()
```



In [5]:

```python
# All columns together: different units, so scale each column to 0-1 (min-max)
fig, ax = plt.subplots(figsize=(12, 6))
for col, label in METRIC_LABELS.items():
    ax.plot(area_df["date"], analysis.minmax_normalize(area_df[col]), label=label)
ax.set_title(f"All columns, min-max normalized ({area_name})")
ax.set_xlabel("Date")
ax.set_ylabel("Normalized value (0 = min, 1 = max)")
ax.legend()
ax.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



2. ENTSO-E: cross-border flows from/to Norway, last two years

In [6]:

```python
# Two full years back from the start of today (UTC)
END = pd.Timestamp.now(tz="UTC").normalize()
START = END - pd.DateOffset(years=2)
TOKEN = SECRETS["entsoe"]["token"]

# Cache on disk so the (slow) API calls are only made once
CACHE = ROOT / "No_sync" / "entsoe_flows.csv"
CACHE.parent.mkdir(exist_ok=True)

if CACHE.exists():
    flows = pd.read_csv(CACHE, parse_dates=["ts"])
else:
    frames = [api.fetch_connection(TOKEN, name, START, END) for name in api.CONNECTIONS]
    flows = pd.concat(frames, ignore_index=True)
    flows.to_csv(CACHE, index=False)

print(flows.shape)
display(flows.groupby(["connection", "direction"]).size().unstack())  # which links returned data
```

```
(315360, 4)
```

direction	export	import
connection		
NO1 - SE3	17520	17520
NO2 - DE_LU	17520	17520
NO2 - DK1	17520	17520
NO2 - GB	17520	17520
NO2 - NL	17520	17520
NO3 - SE2	17520	17520
NO4 - FI	17520	17520
NO4 - SE1	17520	17520
NO4 - SE2	17520	17520

3. Cassandra + Spark: store the flows

In [7]:

```python
import time
print("Cache saved:", time.ctime(CACHE.stat().st_mtime))
print("Share of missing hourly values:", round(100 * flows["mw"].isna().mean(), 1), "%")
```

```
Cache saved: Wed Oct 14 21:35:07 2026
Share of missing hourly values: 0.0 %
```

In [8]:

```python
# Connect to the local Cassandra and create keyspace and table (partition per connection/direction)
from cassandra.cluster import Cluster

KEYSPACE, TABLE = "ind320", "entsoe_flows"
session = Cluster(["localhost"], port=9042).connect()
session.execute(f"CREATE KEYSPACE IF NOT EXISTS {KEYSPACE} WITH REPLICATION = "
                "{'class': 'SimpleStrategy', 'replication_factor': 1};")
session.execute(f"DROP TABLE IF EXISTS {KEYSPACE}.{TABLE};")   # start from scratch on each run
session.execute(f"CREATE TABLE {KEYSPACE}.{TABLE} (connection text, direction text, ts timestamp, "
                "mw double, PRIMARY KEY ((connection, direction), ts));")
```

Out[8]:

```
<cassandra.cluster.ResultSet at 0x154fbb290>
```

In [9]:

```python
import os
os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"          # NEW: keep Spark on localhost

from pyspark.sql import SparkSession, functions as F

spark = (
    SparkSession.builder.appName("IND320Flows")
    .config("spark.driver.bindAddress", "127.0.0.1")
    .config("spark.driver.host", "127.0.0.1")        # NEW: advertise localhost, not mac.lan
    .config("spark.jars.packages", "com.datastax.spark:spark-cassandra-connector_2.12:3.5.1")
    .config("spark.cassandra.connection.host", "127.0.0.1")
    .config("spark.cassandra.connection.port", "9042")
    .config("spark.sql.extensions", "com.datastax.spark.connector.CassandraSparkExtensions")
    .config("spark.sql.session.timeZone", "UTC")     # keep timestamps in UTC
    .getOrCreate()
)
print("Spark", spark.version)
```

```
:: loading settings :: url = jar:file:/Users/ismailnabi/Desktop/Ind%c3%b8k/IND320_Prosjekt/.venv/lib/python3.12/site-packages/pyspark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: /Users/ismailnabi/.ivy2/cache
The jars for the packages stored in: /Users/ismailnabi/.ivy2/jars
com.datastax.spark#spark-cassandra-connector_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-c6fe7321-b40e-4e7e-8c14-28e73381d65a;1.0
	confs: [default]
	found com.datastax.spark#spark-cassandra-connector_2.12;3.5.1 in central
	found com.datastax.spark#spark-cassandra-connector-driver_2.12;3.5.1 in central
	found org.scala-lang.modules#scala-collection-compat_2.12;2.11.0 in central
	found org.apache.cassandra#java-driver-core-shaded;4.18.1 in central
	found com.datastax.oss#native-protocol;1.5.1 in central
	found com.datastax.oss#java-driver-shaded-guava;25.1-jre-graal-sub-1 in central
	found com.typesafe#config;1.4.1 in central
	found org.slf4j#slf4j-api;1.7.26 in central
	found io.dropwizard.metrics#metrics-core;4.1.18 in central
	found org.hdrhistogram#HdrHistogram;2.1.12 in central
	found org.reactivestreams#reactive-streams;1.0.3 in central
	found org.apache.cassandra#java-driver-mapper-runtime;4.18.1 in central
	found org.apache.cassandra#java-driver-query-builder;4.18.1 in central
	found org.apache.commons#commons-lang3;3.10 in central
	found com.thoughtworks.paranamer#paranamer;2.8 in central
	found org.scala-lang#scala-reflect;2.12.19 in central
:: resolution report :: resolve 197ms :: artifacts dl 9ms
	:: modules in use:
	com.datastax.oss#java-driver-shaded-guava;25.1-jre-graal-sub-1 from central in [default]
	com.datastax.oss#native-protocol;1.5.1 from central in [default]
	com.datastax.spark#spark-cassandra-connector-driver_2.12;3.5.1 from central in [default]
	com.datastax.spark#spark-cassandra-connector_2.12;3.5.1 from central in [default]
	com.thoughtworks.paranamer#paranamer;2.8 from central in [default]
	com.typesafe#config;1.4.1 from central in [default]
	io.dropwizard.metrics#metrics-core;4.1.18 from central in [default]
	org.apache.cassandra#java-driver-core-shaded;4.18.1 from central in [default]
	org.apache.cassandra#java-driver-mapper-runtime;4.18.1 from central in [default]
	org.apache.cassandra#java-driver-query-builder;4.18.1 from central in [default]
	org.apache.commons#commons-lang3;3.10 from central in [default]
	org.hdrhistogram#HdrHistogram;2.1.12 from central in [default]
	org.reactivestreams#reactive-streams;1.0.3 from central in [default]
	org.scala-lang#scala-reflect;2.12.19 from central in [default]
	org.scala-lang.modules#scala-collection-compat_2.12;2.11.0 from central in [default]
	org.slf4j#slf4j-api;1.7.26 from central in [default]
	---------------------------------------------------------------------
	|                  |            modules            ||   artifacts   |
	|       conf       | number| search|dwnlded|evicted|| number|dwnlded|
	---------------------------------------------------------------------
	|      default     |   16  |   0   |   0   |   0   ||   16  |   0   |
	---------------------------------------------------------------------
:: retrieving :: org.apache.spark#spark-submit-parent-c6fe7321-b40e-4e7e-8c14-28e73381d65a
	confs: [default]
	0 artifacts copied, 16 already retrieved (0kB/5ms)
26/10/14 21:35:11 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/14 21:35:11 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
Spark 3.5.1
```

3.1 Insert the flows into Cassandra using Spark

In [10]:

```python
import pandas as pd
from pyspark.sql.types import DoubleType, StringType, StructField, StructType, TimestampType

# Explicit schema: no dependence on pandas dtypes (pandas 3 + PySpark 3.5 clash)
schema = StructType([
    StructField("connection", StringType(), False),
    StructField("direction", StringType(), False),
    StructField("ts", TimestampType(), False),
    StructField("mw", DoubleType(), True),
])

clean = flows.dropna(subset=["mw"]).copy()
clean["ts"] = pd.to_datetime(clean["ts"])
if clean["ts"].dt.tz is None:
    clean["ts"] = clean["ts"].dt.tz_localize("UTC")      # mark as UTC so Spark does not shift hours

rows = list(zip(clean["connection"], clean["direction"],
                clean["ts"].dt.to_pydatetime(), clean["mw"].astype(float)))
print("rows to write:", len(rows))

# Write in batches of 50 000 rows through Spark (partition key makes re-runs idempotent)
for i in range(0, len(rows), 50_000):
    (spark.createDataFrame(rows[i:i + 50_000], schema).write.format("org.apache.spark.sql.cassandra")
        .options(table=TABLE, keyspace=KEYSPACE).mode("append").save())
    print(f"written {min(i + 50_000, len(rows))} / {len(rows)}")
```

```
rows to write: 315360
written 50000 / 315360
written 100000 / 315360
written 150000 / 315360
written 200000 / 315360
written 250000 / 315360
written 300000 / 315360
written 315360 / 315360
```

3.2 Data quality: missing hourly values per connection

In [11]:

```python
# How many hourly values are missing per connection and direction?
summary = (flows.assign(missing=flows["mw"].isna())
           .groupby(["connection", "direction"])
           .agg(rows=("mw", "size"), missing=("missing", "sum"),
                first=("ts", "min"), last=("ts", "max")))
summary["missing_pct"] = (100 * summary["missing"] / summary["rows"]).round(1)
display(summary)
```

		rows	missing	first	last	missing_pct
connection	**direction**					
NO1 - SE3	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO2 - DE_LU	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO2 - DK1	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO2 - GB	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO2 - NL	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO3 - SE2	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO4 - FI	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO4 - SE1	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
NO4 - SE2	**export**	17520	0	2024-10-14	2026-10-13 23:00:00	0.0
	import	17520	0	2024-10-14	2026-10-13 23:00:00	0.0

3.3 Read back with Spark and curate the data

In [12]:

```python
import os, time
from pyspark.sql import functions as F

# Table location in Cassandra (defined here so the cell works on its own)
KEYSPACE, TABLE = "ind320", "entsoe_flows"

# Spark returns timestamps in the computer's local time zone: force UTC
os.environ["TZ"] = "UTC"
time.tzset()

# Read back with Spark and extract the relevant columns
stored = (spark.read.format("org.apache.spark.sql.cassandra")
          .options(table=TABLE, keyspace=KEYSPACE).load())
print("Rows in Cassandra:", stored.count())

# Curate: one document per connection, direction, year and month
# (total energy for the pie chart + the hourly series for the line plot)
monthly = (
    stored.withColumn("year", F.year("ts")).withColumn("month", F.month("ts"))
    .groupBy("connection", "direction", "year", "month")
    .agg(F.sum("mw").alias("total_mwh"),                          # hourly MW summed = MWh
         F.sort_array(F.collect_list(F.struct("ts", "mw"))).alias("series"))
)

# collect() instead of toPandas(): avoids the distutils/pandas 3 incompatibility
monthly_pd = pd.DataFrame([r.asDict() for r in monthly.collect()])
monthly_pd = monthly_pd.sort_values(["connection", "direction", "year", "month"]).reset_index(drop=True)
monthly_pd.drop(columns="series").head()
```

```
Rows in Cassandra: 315360
```

Out[12]:

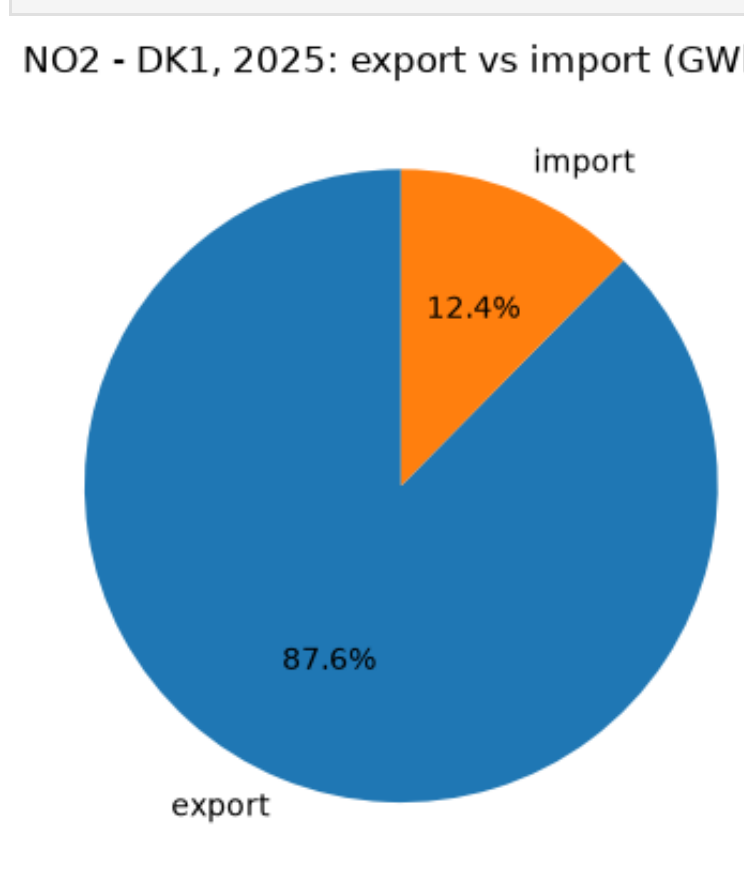
	connection	direction	year	month	total_mwh
0	NO1 - SE3	export	2024	10	6543.93
1	NO1 - SE3	export	2024	11	80039.10
2	NO1 - SE3	export	2024	12	176925.59
3	NO1 - SE3	export	2025	1	210883.15
4	NO1 - SE3	export	2025	2	240851.79

4. Plots from the Spark-extracted data

In [13]:

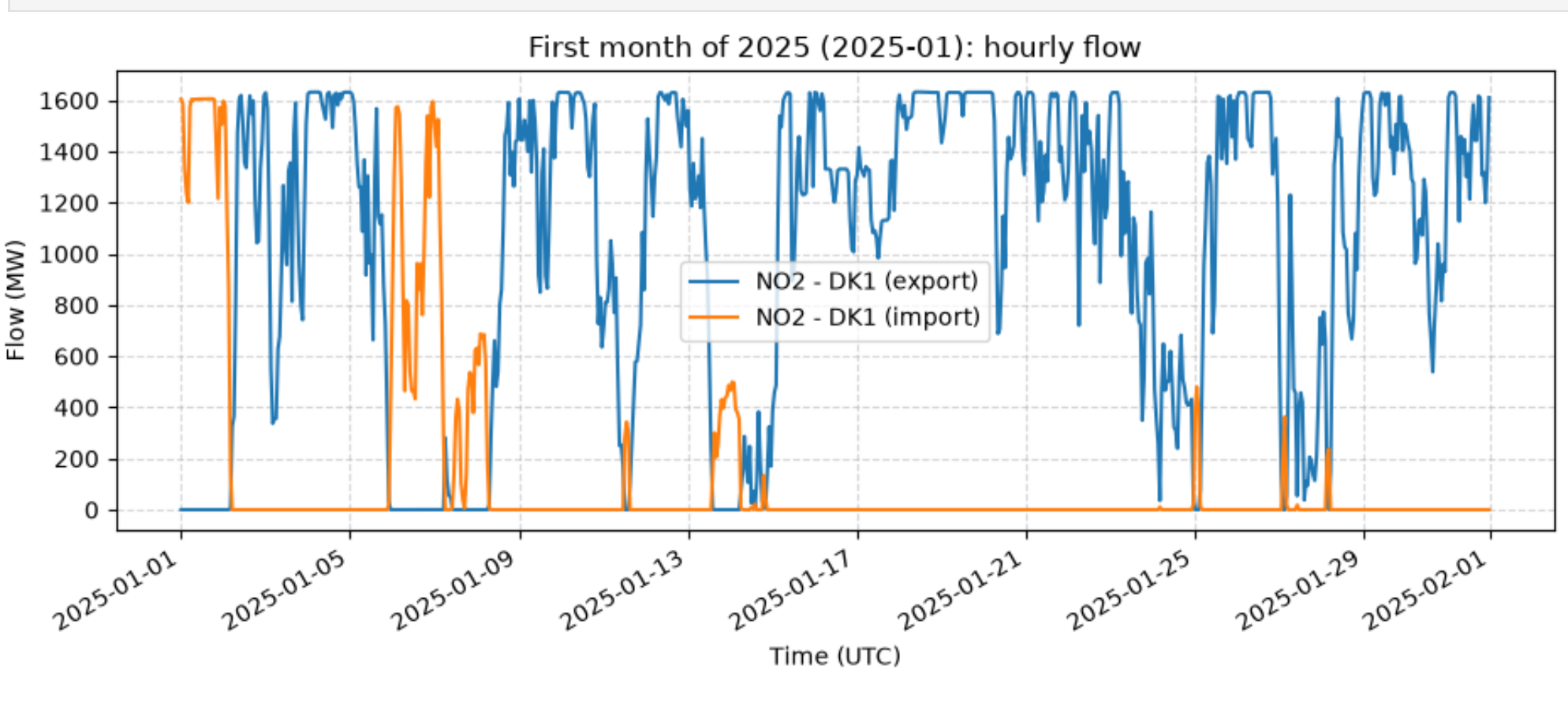
```python
# Pie chart: total energy in and out for one connection and one full year
CONNECTION, YEAR = "NO2 - DK1", int(monthly_pd["year"].value_counts().idxmax())
sel = monthly_pd[(monthly_pd["connection"] == CONNECTION) & (monthly_pd["year"] == YEAR)]
totals = sel.groupby("direction")["total_mwh"].sum() / 1000     # GWh

plt.pie(totals, labels=totals.index, autopct="%1.1f%%", startangle=90)
plt.title(f"{CONNECTION}, {YEAR}: export vs import (GWh)")
plt.show()
```



In [14]:

```python
# Line plot: first month of the year, separate lines for each connection and direction.
# Add more names to PLOT_CONNECTIONS to compare several connections in the same plot.
PLOT_CONNECTIONS = [CONNECTION]
first_month = sel["month"].min()
fig, ax = plt.subplots(figsize=(11, 4))
month_rows = monthly_pd[(monthly_pd["connection"].isin(PLOT_CONNECTIONS))
                        & (monthly_pd["year"] == YEAR) & (monthly_pd["month"] == first_month)]
for _, row in month_rows.iterrows():
    ts = [r["ts"] for r in row["series"]]
    mw = [r["mw"] for r in row["series"]]
    ax.plot(ts, mw, label=f"{row['connection']} ({row['direction']})")
ax.set_title(f"First month of {YEAR} ({YEAR}-{first_month:02d}): hourly flow")
ax.set_xlabel("Time (UTC)"); ax.set_ylabel("Flow (MW)")
ax.legend(); ax.grid(True, linestyle="--", alpha=0.5)
fig.autofmt_xdate()
plt.show()
```



5. MongoDB: store the Spark-curated data

In [15]:

```python
# Build documents: arrays of timestamps and values keep the collection small (~400 documents)
docs = [
    {"connection": r.connection, "direction": r.direction, "year": int(r.year), "month": int(r.month),
     "total_mwh": float(r.total_mwh),
     "ts": [x["ts"] for x in r.series], "mw": [float(x["mw"]) for x in r.series]}
    for r in monthly_pd.itertuples()
]

m = SECRETS["mongo"]
client = db.get_client(db.build_uri(m["user"], m["password"], m["host"]))
client.admin.command("ping")                          # test the connection
collection = client[m["database"]]["entsoe_flows_monthly"]
collection.delete_many({})                            # idempotent: replace previous content
collection.insert_many(docs)
print("Documents in MongoDB:", collection.count_documents({}))
```

```
Documents in MongoDB: 450
```

6. Log and AI usage

Part 2 replaced the CSV file with live data and added two databases. I reused the repository and Streamlit app from part 1 and work on a separate branch until peer review is finished.

I started with the NVE Magasinstatistikk API, which needs no key. It has four endpoints; I use HentOffentligData for weekly values since 1995 and HentOmråder for area names. The areas are NO (the whole country), EL (five price areas) and VASS (four watercourse areas). My part 1 experience with the time columns helped: the data has one row per week and area, so my earlier plots mixed several areas in one line, and the app looked for a date column that is actually called dato_Id. Dates are now parsed properly and every plot shows one area, or one line per area. My first area-name function failed because the API returned a list instead of the nested structure in the documentation, so it now handles both shapes.

For ENTSO-E I created an account, got a security token and fetched physical cross-border flows for nine connections, both directions, for two years. About half of the hourly values were missing. Forward-filling omitted points did not change that. Instead, the two directions of a cable almost never had data in the same hour, so I assumed a missing hour in one direction means zero flow when the other direction is reported. Very few hours were missing in both directions, and afterwards Cassandra held 315,360 rows, exactly 9 x 2 x 17,520 hours.

The data went into Cassandra (in Docker) through Spark, was read back with Spark and aggregated to one document per connection, direction, year and month. These 450 documents are stored in MongoDB Atlas. The setup took the most time: a MongoDB authentication error (solved with a new password), Spark advertising my computer's hostname instead of localhost (fixed with spark.driver.host), and PySpark 3.5 clashing with pandas 3 and Python 3.12, which I avoided with an explicit schema and collect() instead of toPandas().

In Streamlit I rebuilt the pages. The table and plot pages use st.radio for area type and a select slider for the month interval, and the all-columns plot is min-max normalised for one area. The flow page has two columns with a pie chart (radio and year), a line plot (st.pills for months) and an expander documenting the source. Credentials live in No_sync/secrets.toml locally and in Streamlit secrets online, never on GitHub. Code repeated in part 1 now sits in the modules folder.

AI usage

I used Claude (Anthropic) as a programming assistant throughout part 2, and Gemini and Copilot earlier in the course. The AI helped me to:

Design the project structure and move repeated code into shared modules (helpers, API access, analysis, database access). Write the first versions of the NVE and ENTSO-E API code, the Spark and Cassandra cells, the MongoDB code and the Streamlit pages. Debug errors I ran into: the NVE response format, MongoDB authentication, the Spark hostname problem, the PySpark and pandas incompatibility, the missing ENTSO-E hours and a duplicate page name in Streamlit.

I ran all code myself, read the error messages and results, and decided on the data handling choices, such as the zero-flow assumption. I verified results by checking row counts (for example 315,360 rows in Cassandra) and that the plots matched the data. Passwords and API tokens were kept out of the AI conversation and out of GitHub.